

# CS 412 Group Project Final Report

Ayhan Salih Öner, Sezer Enis Birgili, Emir Kavukcu, Zeynep Kılıçarslan, Eda Pekol  
Team: Predictive Pioneers

## Introduction

This project seeks to address a categorization challenge by assigning severity levels to bug descriptions. The severity ratings are as follows: Enhancement (1), Minor (2), Normal (3), Major (4), Blocker (5), and Critical (6), ranging from 1 to 6. The research employs several machine learning algorithms, with models evaluated based on macro precision. Accurately identifying each bug report's severity score requires reviewing bug reports, which give descriptions and bug kinds.

## Problem Description

Sorting bug complaints into six severity categories is the work at hand. The objective is to precisely assign a severity score based on the description and bug type in each bug report. This classification problem is critical for prioritizing issue repairs and boosting software maintenance methods. Given the nature of the challenge, the models must handle textual input properly and be resilient to class imbalances in the dataset.

## Methods

In tackling this problem, multiple data pretreatment processes were applied. The raw data was first cleaned to remove any discrepancies and missing values. Text preprocessing techniques such as tokenization, stop-word removal, and stemming/lemmatization were tested and employed to prepare the textual data. For feature extraction, TF-IDF (Term Frequency-Inverse Document Frequency) and n-gram features were utilized to convert the text input into numerical features appropriate for machine learning models. The BERT tokenizer was also utilized to leverage pre-trained language models for feature extraction.

To handle class imbalance, approaches like SMOTE (Synthetic Minority Over-sampling Technique), ROS (Random Over-Sampling), and RUS (Random Under-Sampling) were applied in several tests. Various machine learning models were constructed and assessed, including Random Forest (RF), XGBoost, AdaBoost,

LightGBM (LGBM), logistic regression, SVM, and Convolutional Neural Networks (CNN). The Random Forest model with different boosting techniques were tried with both with and without SMOTE to address class imbalance. All of these models were subjected to substantial hyperparameter tuning using grid search, and the neural network model using focal loss was tested instead of cross-entropy for improved handling of class imbalance. Additionally, stacking was employed to aggregate predictions from multiple models to increase overall performance. Stratified K-Fold cross-validation was utilized to achieve respective assessments across all models, using macro precision as the evaluation criterion.

## **Results and Discussion**

As stated in the methods, numerous experiments were conducted to evaluate the effectiveness of different models and preprocessing techniques. Logistic regression was the first model tested, with the best hyperparameters identified through grid search. The implementation of SMOTE provided superior outcomes in handling imbalanced classes. Additionally, a neural network incorporating focal loss was tested to handle class imbalance more successfully. Beyond these, Random Forest models, both with and without boosting techniques, were extensively experimented with. For instance, LightGBM was assessed under various settings, including with and without SMOTE and light preprocessing techniques. SMOTE and mild preprocessing considerably enhanced the model's performance. Similarly, XGBoost and AdaBoost produced competitive performance metrics. SVMs were also investigated but did not outperform the ensemble algorithms. Based on these experiments, it was concluded that Random Forest without boosting was the best choice for achieving maximum macro precision scores.

After these experiments, the inclusion of a BERT tokenizer for feature extraction was tested with the best model, which was Random Forest, resulting in superior model performance in terms of macro precision. Therefore, continuous experiments with grid search were conducted using the BERT tokenizer for feature extraction to find the best hyperparameters. Ultimately, the best macro precision score achieved on the public part of the test data was 0.68, using the parameters “criterion=gini, max\_depth=None, min\_samples\_leaf=2, min\_samples\_split=10”. However, despite achieving 0.68 on the public part, this prediction got a macro precision score of 0.55 on the private part of the test data, which was unfortunate for us since other predictions utilizing different hyperparameters with the same model got scores around 0.65-0.68 on the private part, much closer to the maximum score we achieved. The best model we developed was tested with 3-fold cross-validation on the training data. Below is the best classification report we obtained in this testing, which was on the 3rd fold.

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.80      | 0.01   | 0.02     | 401     |
| 1            | 0.65      | 0.01   | 0.03     | 1475    |
| 2            | 0.75      | 0.01   | 0.01     | 1034    |
| 3            | 0.86      | 0.98   | 0.91     | 41951   |
| 4            | 0.76      | 0.04   | 0.08     | 2018    |
| 5            | 0.64      | 0.03   | 0.06     | 234     |
| 6            | 0.79      | 0.68   | 0.73     | 6219    |
| accuracy     |           |        | 0.85     | 53332   |
| macro avg    | 0.75      | 0.25   | 0.26     | 53332   |
| weighted avg | 0.84      | 0.85   | 0.81     | 53332   |

### *Classification Report on Fold 3*

Additional metrics, such as confusion matrices, ROC curves and Precision-Recall curves obtained from testing on the training data for each fold can be seen in the appendix.

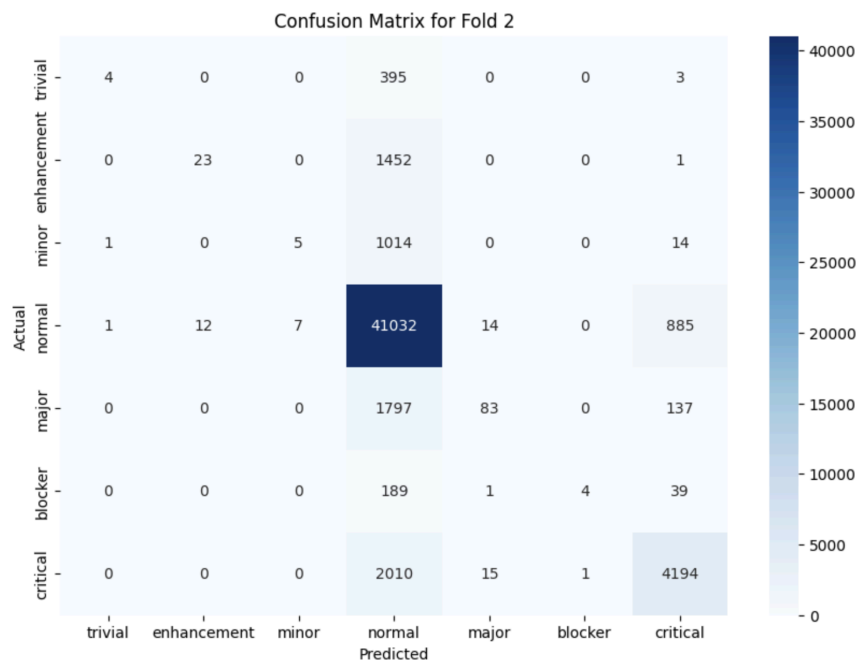
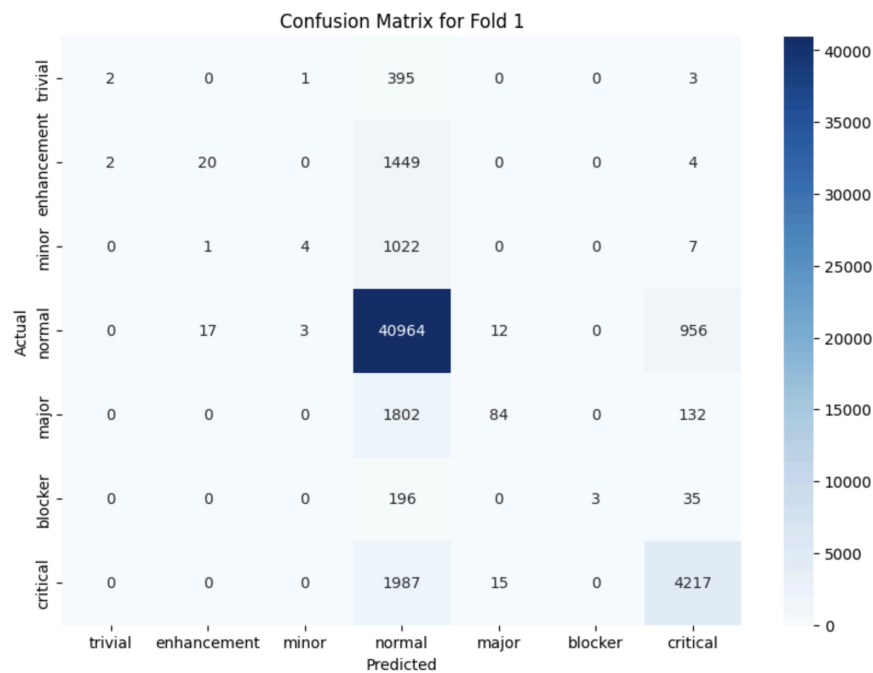
## **Conclusion**

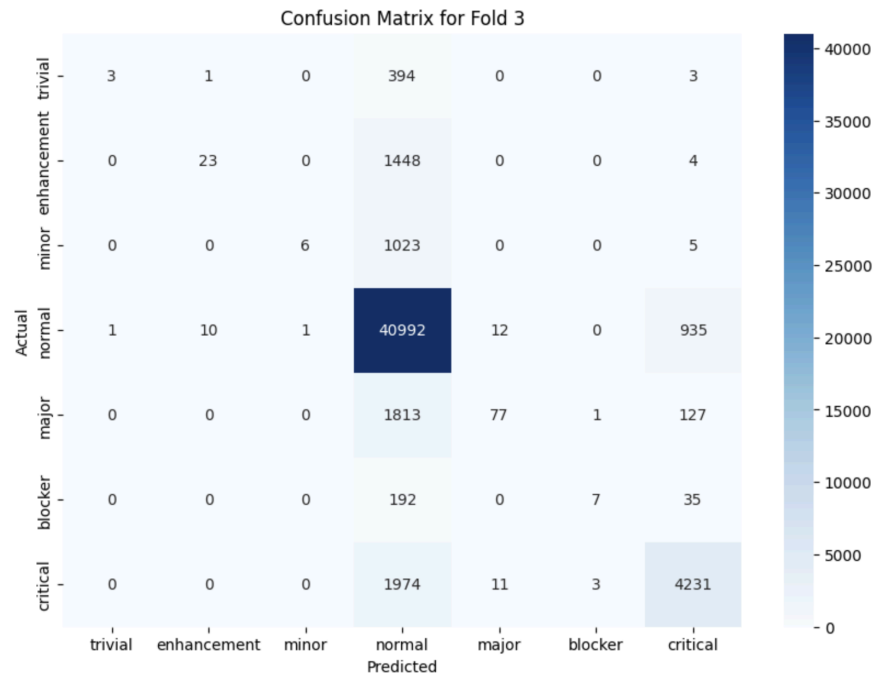
This project effectively demonstrated the application of machine learning techniques to classify bug severity. An ensemble approach, Random Forest, paired with SMOTE for addressing class imbalance and BERT tokenizer for feature extraction, produced the best results. This model got a significant public score of 0.68861 and a private score of 0.55158. Overall, the project underlined the significance of preprocessing, feature extraction and controlling class imbalance to improve model performance.

## **Appendix**

Ayhan, Emir, and Sezer primarily worked on the coding and implementation aspects of the project, including data preprocessing, model training, and evaluation. Zeynep and Eda focused mainly on the report writing and documentation, ensuring that the project's findings and methodologies were clearly communicated.

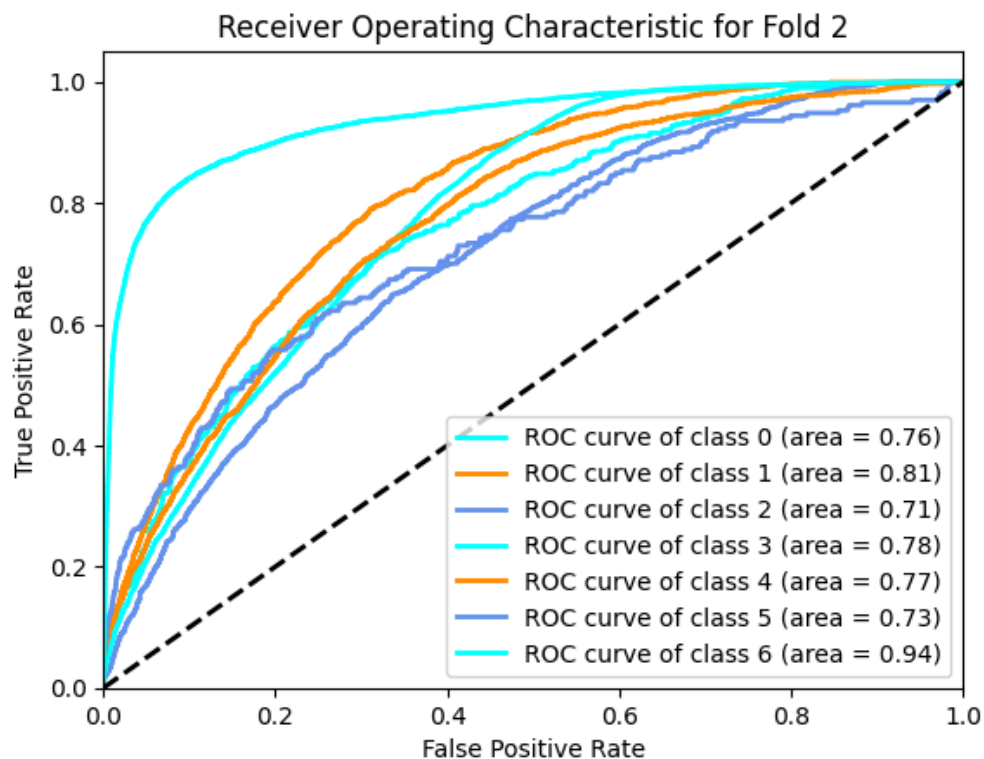
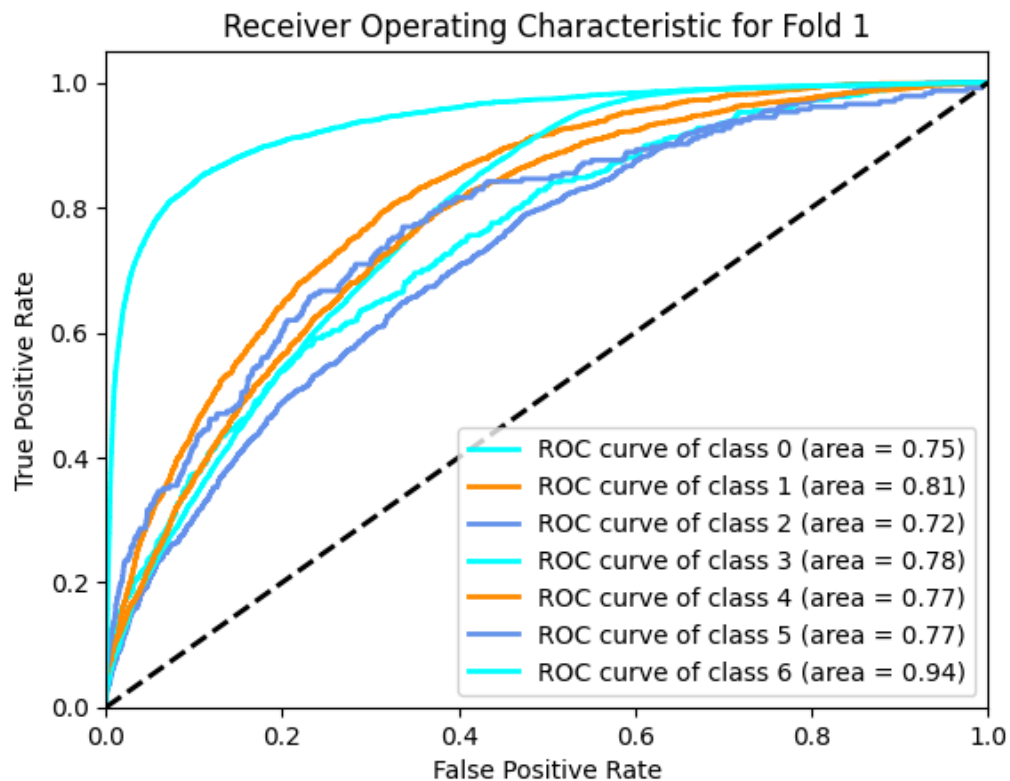
# Confusion Matrices

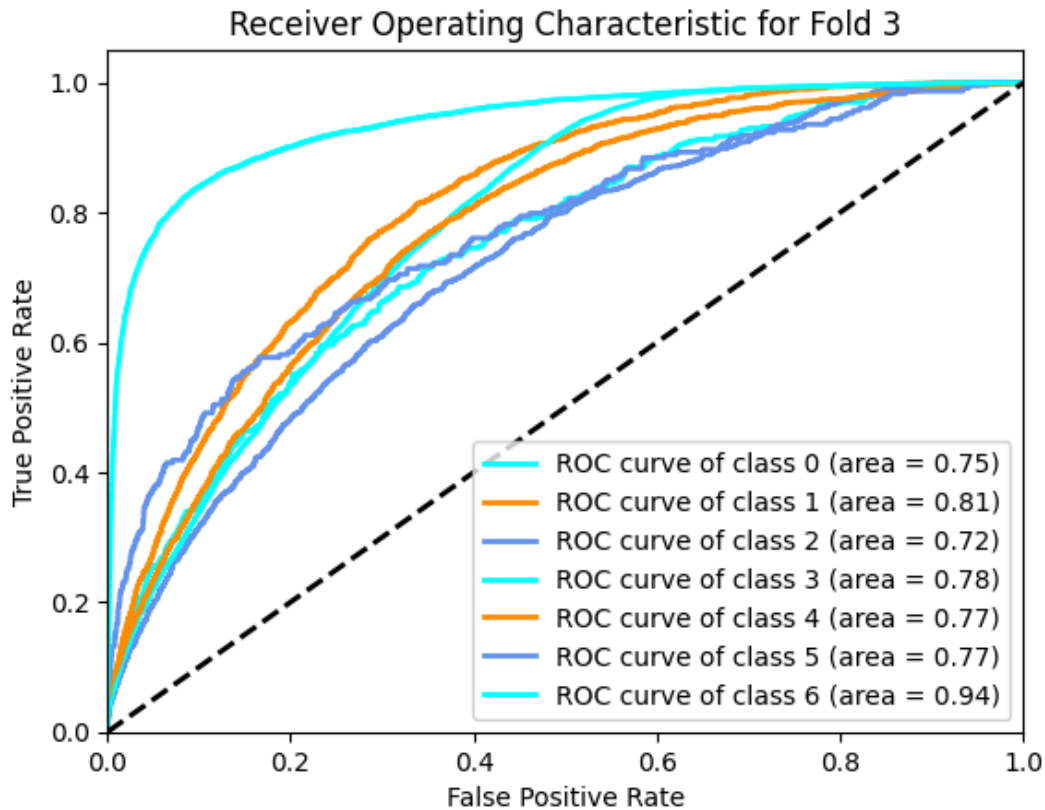




On the confusion matrices, the normal class has the highest correct prediction rates, as seen from the large value on the diagonal corresponding to this class. Misclassifications are usually prominent in minority classes. For example, instances of trivial and minor classes are often misclassified into other categories, highlighting the importance and the challenge of handling the class imbalances.

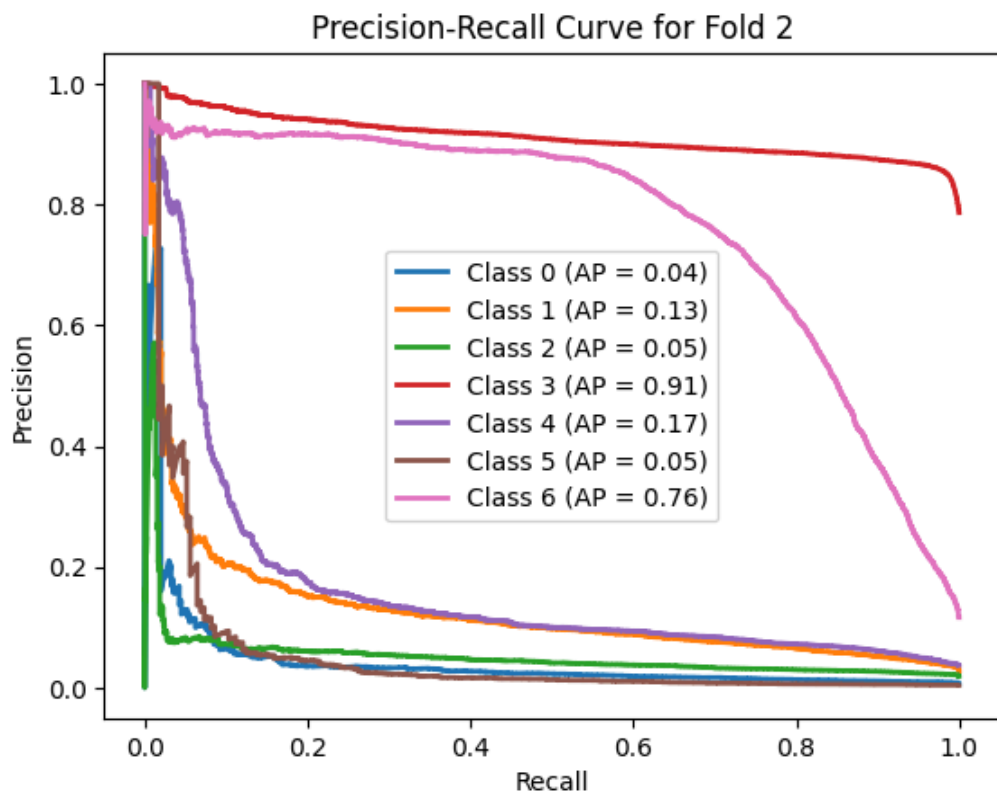
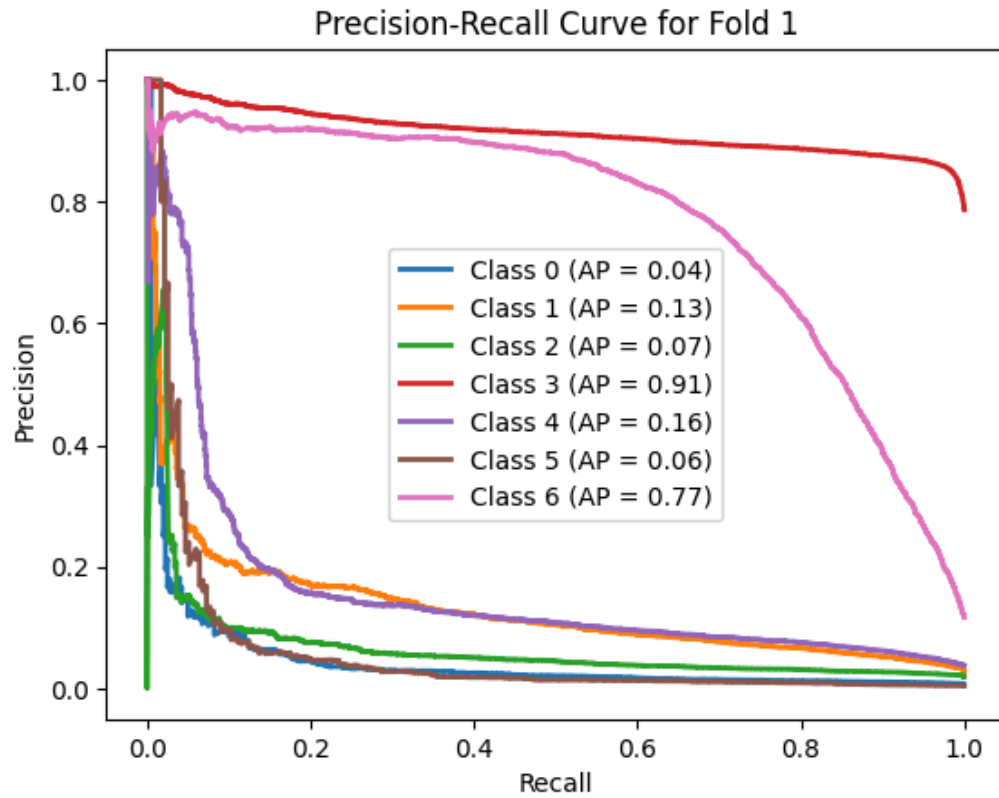
# ROC Curves

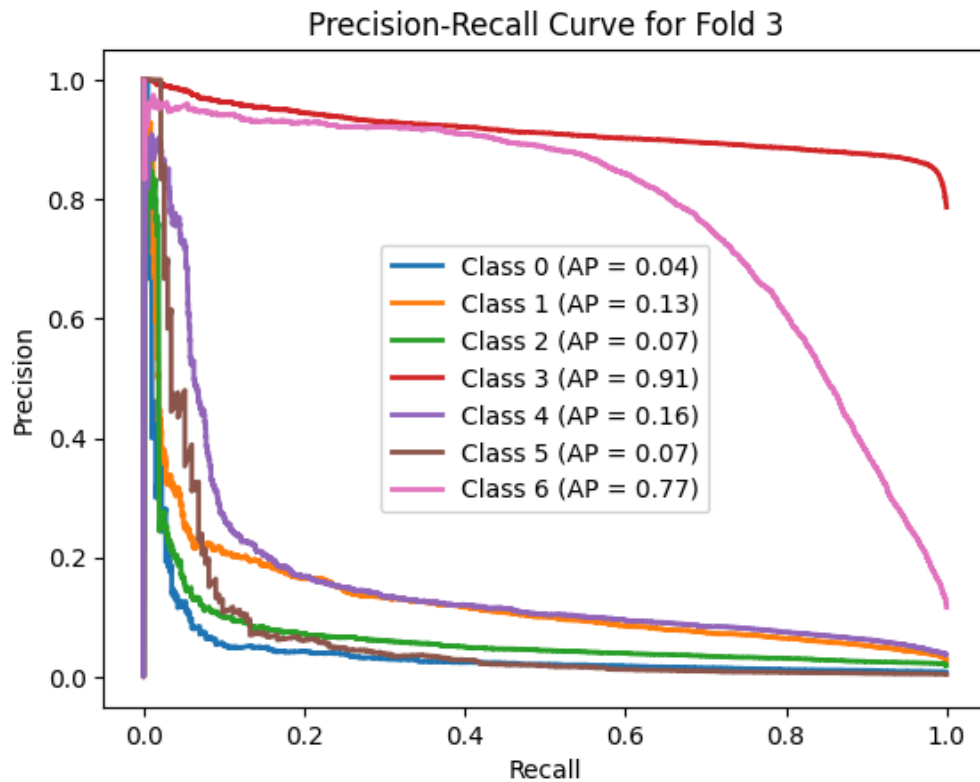




The Receiver Operating Characteristic (ROC) curves for each fold show the true positive rate (sensitivity) vs. false positive rate (1-specificity) for each severity class. The area under the curve (AUC) provides a single metric to compare model performance across different classes. Higher AUC values indicate better model performance. The ROC curves demonstrate how well the model distinguishes between classes. For example, the high AUC for the "critical" class indicates strong performance in identifying critical severity bugs, while lower AUC values for other classes suggest areas for improvement.

## Precision - Recall Curves





The Precision-Recall (PR) curves for each fold plot precision vs. recall for each severity class. These curves are particularly useful for evaluating model performance on imbalanced datasets. The Average Precision (AP) score summarizes the precision-recall trade-off. Higher AP values indicate better performance. The PR curves provide insights into how well the model maintains precision at various levels of recall. This is important for understanding model performance in identifying minority classes, where maintaining high precision can be challenging.